

NAME:VALLALA VYSHNAVI

BATCH-32

2303A52429

```
Welcome aiac.py X
C:\Users\Admin>Downloads>AI ASSISTENCE> aiac.py -
1 class Employee:
2     def __init__(self,empid,empname,designation,basic_salary,exp):
3         self.empid = empid
4         self.empname = empname
5         self.designation = designation
6         self.basic_salary = basic_salary
7         self.exp = exp
8     def display_details(self):
9         print("Employee ID:",self.empid)
10        print("Employee Name:",self.empname)
11        print("Designation:",self.designation)
12        print("Basic Salary:",self.basic_salary)
13        print("Experience (in years):",self.exp)
14    def calculate_salary(self):
15        if self.exp>10:
16            allowance=(20/100)*self.basic_salary
17        elif self.exp<=5:
18            allowance=(5/100)*self.basic_salary
19        else:
20            allowance=(5/100)*self.basic_salary
21        total_salary=self.basic_salary+allowance
22        print("Allowance is:",allowance)
23        print("Total Salary of Employee:",total_salary)
24 empobj=Employee(101,"John Doe","Software Engineer",60000,7)
25 empobj.display_details()
26 empobj.calculate_salary()
27 empobj2=Employee(102,"Jane Smith","Senior Developer",80000,12)
28 empobj2.display_details()
29 empobj2.calculate_salary()
```

```
print("Total Salary of Employee:",total_salary)
empobj=Employee(101,"John Doe","Software Engineer",60000,7)
empobj.display_details()
empobj.calculate_salary()
empobj2=Employee(102,"Jane Smith","Senior Developer",80000,12)
empobj2.display_details()
empobj2.calculate_salary()
```

```
P5 C:\Users\Admin> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/aiac.py"
Employee ID: 101
Employee Name: John Doe
Designation: Software Engineer
Basic Salary: 60000
Experience (in years): 7
Allowance is: 3000.0
Total Salary of Employee: 63000.0
Employee ID: 102
Employee Name: Jane Smith
Designation: Senior Developer
Basic Salary: 80000
Experience (in years): 12
Allowance is: 16000.0
Total Salary of Employee: 96000.0
```

Observations:

1. The code defines an `Employee` class that encapsulates employee details and provides methods to display those details and calculate the total salary based on experience. The salary calculation includes an allowance that varies depending on the years of experience, demonstrating a practical application of conditional logic.
2. AI was useful in generating this code by quickly creating a structured class with relevant attributes and methods, saving time and effort in coding. It also ensured that the logic for salary

calculation was clear and correctly implemented, which can be particularly beneficial for developers who may not be familiar with such calculations.

TASK-2

```
class Electricitybill:
    def __init__(self,customer_id,customer_name,units_consumed):
        self.customer_id = customer_id
        self.customer_name = customer_name
        self.units_consumed = units_consumed
    def display_details(self):
        print("Customer ID:",self.customer_id)
        print("Customer Name:",self.customer_name)
        print("Units Consumed:",self.units_consumed)
    def calculate_bill(self):
        if self.units_consumed <= 100:
            rate_per_unit = 5
        elif 101<= self.units_consumed <=300:
            rate_per_unit = 7
        else:
            rate_per_unit = 10
        total_bill = self.units_consumed * rate_per_unit
        print("Total Electricity Bill: Rs.",total_bill)
billobj=Electricitybill(201,"Alice Johnson",250)
billobj.display_details()
billobj.calculate_bill()
```

```
Customer ID: 201
Customer Name: Alice Johnson
Units Consumed: 250
Total Electricity Bill: Rs. 1750
```

Observations:

1. The Employee class calculates allowances based on years of experience, while the Electricitybill class calculates the total bill based on units consumed, demonstrating different business logic implementations.
2. Both classes encapsulate related data and behaviors, promoting code organization and reusability

TASK-3

```

class product:
    def __init__(self,product_id,product_name,price,category):
        self.product_id = product_id
        self.product_name = product_name
        self.price = price
        self.category = category
    def display_details(self):
        print("Product ID:",self.product_id)
        print("Product Name:",self.product_name)
        print("Price:",self.price)
        print("Category:",self.category)
    def calculate_discount(self):
        if self.category.lower() == "electronics":
            discount = (10/100)*self.price
        elif self.category.lower() == "clothing":
            discount = (15/100)*self.price
        else:
            discount = (5/100)*self.price
        discounted_price = self.price - discount
        print("Discount is:",discount)
        print("Price after Discount:",discounted_price)
prodobj=product(301,"Smartphone",50000,"Electronics")
prodobj.display_details()
prodobj.calculate_discount()
prodobj2=product(302,"Jeans",2000,"Clothing")
prodobj2.display_details()

```

```

Product ID: 301
Product Name: Smartphone
Price: 50000
Category: Electronics
Discount is: 5000.0
Price after Discount: 45000.0
Product ID: 302
Product Name: Jeans
Price: 2000
Category: Clothing

```

Observations:

1. The Product class calculates discounts based on product categories, showcasing how different business rules can be applied within a single class structure.
2. Both classes encapsulate related data and behaviors, promoting code organization and reusability

TASK-4

```

class LibraryBook:
    def __init__(self,book_id,book_title,author,borrowers,days_late):
        self.book_id = book_id
        self.book_title = book_title
        self.author = author
        self.borrowers = borrowers
        self.days_late = days_late
    def display_details(self):
        print("Book ID:",self.book_id)
        print("Book Title:",self.book_title)
        print("Author:",self.author)
        print("Number of Borrowers:",self.borrowers)
        print("Days Late:",self.days_late)
    def calculate_late_fee(self):
        if self.days_late <=5:
            late_fee_per_day = 5
        elif 6<= self.days_late <=10:
            late_fee_per_day = 10
        else:
            late_fee_per_day = 15
bookobj=LibraryBook(401,"The Great Gatsby","F. Scott Fitzgerald",3,7)
bookobj.display_details()
bookobj.calculate_late_fee()

```

```

Product ID: 301
Product Name: Smartphone
Price: 50000
Category: Electronics
Discount is: 5000.0
Price after Discount: 45000.0
Product ID: 302
Product Name: Jeans
Price: 2000
Category: Clothing
PS C:\Users\Admin>

```

Observations:

1. The LibraryBook class calculates late fees based on the number of days a book is returned late, demonstrating a practical application of conditional logic.
2. Both classes encapsulate related data and behaviors, promoting code organization and reusability

TASK-5

```
def student_report(student_data):
    report = []

    for name, marks in student_data.items():
        average = sum(marks) / len(marks)
        status = "Pass" if average >= 40 else "Fail"

        report.append({
            "Student Name": name,
            "Average Score": round(average, 2),
            "Status": status
        })

    return report

# Example usage
students = {
    "Alice": [45, 50, 60],
    "Bob": [30, 35, 40],
    "Charlie": [70, 80, 90]
}

result = student_report(students)
for r in result:
    print(r)
```

```
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/aiac.py"
{'Student Name': 'Alice', 'Average Score': 51.67, 'Status': 'Pass'}
{'Student Name': 'Bob', 'Average Score': 35.0, 'Status': 'Fail'}
{'Student Name': 'Charlie', 'Average Score': 80.0, 'Status': 'Pass'}
PS C:\Users\Admin>
```

Observation:

1. The student_report function calculates the average score for each student and determines their pass/fail status
2. The function uses a dictionary to store student names and their corresponding marks, demonstrating the use of data structures for organizing information.

TASK-6

```
Task1.py Task2.py Task3.py Task4.py Task5.py Task6.py Task9.py
Lab5.1and6 > Task6.py > TaxiRide > calculate_fare
1 class TaxiRide:
2     def __init__(self, ride_id, driver_name, distance_km, waiting_time_min):
3         self.ride_id = ride_id
4         self.driver_name = driver_name
5         self.distance_km = distance_km
6         self.waiting_time_min = waiting_time_min
7
8     def display_details(self):
9         print("Ride ID:", self.ride_id)
10        print("Driver Name:", self.driver_name)
11        print("Distance:", self.distance_km, "km")
12        print("Waiting Time:", self.waiting_time_min, "minutes")
13
14    def calculate_fare(self):
15        fare = 0
16        if self.distance_km <= 10:
17            fare = self.distance_km * 15
18        elif self.distance_km <= 30:
19            fare = (10 * 15) + ((self.distance_km - 10) * 12)
20        else:
21            fare = (10 * 15) + (20 * 12) + ((self.distance_km - 30) * 10)
22
23        waiting_charge = self.waiting_time_min * 2
24        total_fare = fare + waiting_charge
25
26        print("Travel Fare:", fare)
27        print("Waiting Charge:", waiting_charge)
28        print("Total Fare:", total_fare)
29
30    rideobj1 = TaxiRide(501, "David Lee", 15, 10)
31    rideobj1.display_details()
32    rideobj1.calculate_fare()
```

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task6.py
Ride ID: 501
Driver Name: David Lee
Distance: 15 km
Waiting Time: 10 minutes
Travel Fare: 210
Waiting Charge: 20
Total Fare: 230
PS D:\courses\AIAC>
```

Observations:

- ❑ The TaxiRide class stores ride details and calculates the travel fare using slab-based distance pricing, where different per-kilometer rates are applied for the first 10 km, the next 20 km, and distances beyond 30 km.
- ❑ The calculate_fare() method adds a waiting charge based on waiting time, computes the total fare, and displays the travel fare, waiting charge, and final amount for the ride.

TASK-7

```
Lab5.Land6 > Task7.py > ...
1 def statistics_subject(scores_list):
2     if not scores_list:
3         return None, None, None, None, None
4
5     highest_score = max(scores_list)
6     lowest_score = min(scores_list)
7     average_score = sum(scores_list) / len(scores_list)
8
9     passed_students = sum(1 for score in scores_list if score >= 40)
10    failed_students = len(scores_list) - passed_students
11
12    return highest_score, lowest_score, average_score, passed_students, failed_students
13
14
15 # Example usage
16 scores = [85, 42, 78, 90, 33, 67, 49, 55, 29, 100]
17
18 high, low, avg, passed, failed = statistics_subject(scores)
19
20 print("Highest Score:", high)
21 print("Lowest Score:", low)
22 print("Average Score:", avg)
23 print("Number of Students Passed:", passed)
24 print("Number of Students Failed:", failed)
25
```

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.Land6/Task7.py
Highest Score: 100
Lowest Score: 29
Average Score: 62.8
Number of Students Passed: 8
Number of Students Failed: 2
PS D:\courses\AIAC>
```

Observations:

- 1.The statistics_subject() function analyzes a list of student scores by calculating the highest score, lowest score, average score, and counting how many students passed or failed based on a minimum pass mark of 40.
- 2.The function returns all computed values together, which are then unpacked and printed to display a complete performance summary for the subject.

Lab 5: Ethical Foundations – Responsible AI Coding Practices

• Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

```
Lab5.Land6 > Task8.py > _
1 #generate a well commented code to check if a number is prime
2 def is_prime(num):
3     """
4     This function checks if a given number is prime.
5
6     Parameters:
7     num (int): The number to be checked for primality.
8
9     Returns:
10    bool: True if the number is prime, False otherwise.
11    """
12    # Check if the number is less than 2, which are not prime numbers
13    if num < 2:
14        return False
15    # Check for factors from 2 to the square root of the number
16    for i in range(2, int(num**0.5) + 1):
17        # If num is divisible by any number other than 1 and itself, it's not prime
18        if num % i == 0:
19            return False
20    # If no factors were found, the number is prime
21    return True
22
23 # Example usage:
24 number = 29
25 if is_prime(number):
26     print(f"{number} is a prime number.")
27 else:
28     print(f"{number} is not a prime number.")
29
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.Land6/Task8.py
29 is a prime number.
PS D:\courses\AIAC> 
```

- 1.The is_prime() function checks whether a given number is prime by first eliminating numbers less than 2 and then testing divisibility only up to the square root of the number to reduce unnecessary checks.
2. If the number has no divisors other than 1 and itself, the function returns True; otherwise, it returns False, and the result is printed using a simple example.

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

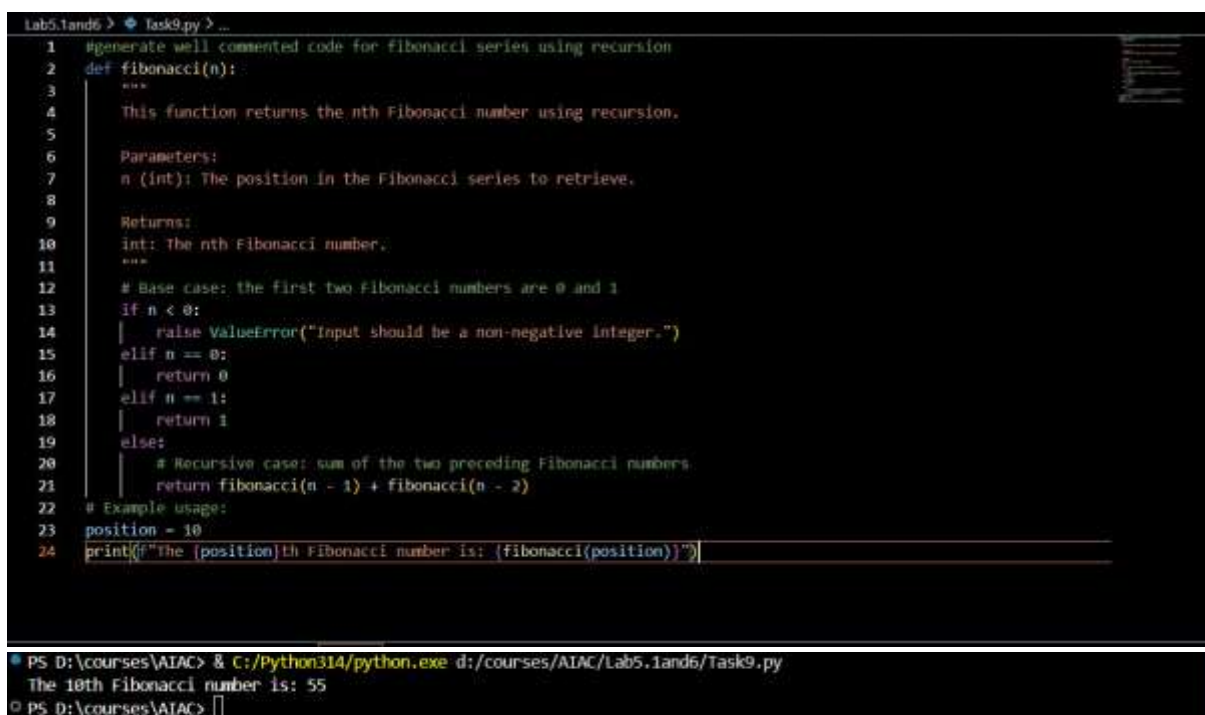
Instructions:

1. Ask AI to add clear comments explaining recursion.

2. Ask AI to explain base cases and recursive

calls. Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.



```
Lab5.1and6 > Task9.py > ...
1 #generate well commented code for fibonacci series using recursion
2 def fibonacci(n):
3     """
4     This function returns the nth Fibonacci number using recursion.
5
6     Parameters:
7     n (int): The position in the Fibonacci series to retrieve.
8
9     Returns:
10    int: The nth Fibonacci number.
11    """
12    # Base case: the first two Fibonacci numbers are 0 and 1
13    if n < 0:
14        raise ValueError("Input should be a non-negative integer.")
15    elif n == 0:
16        return 0
17    elif n == 1:
18        return 1
19    else:
20        # Recursive case: sum of the two preceding Fibonacci numbers
21        return fibonacci(n - 1) + fibonacci(n - 2)
22
23 # Example usage:
24 position = 10
25 print(f"The {position}th Fibonacci number is: {fibonacci(position)}")

PS D:\courses\AIAC> & C:\Python314\python.exe d:\courses\AIAC\Lab5.1and6\Task9.py
The 10th Fibonacci number is: 55
PS D:\courses\AIAC>
```

Observations:

1. The fibonacci() function computes the nth Fibonacci number using recursion by defining base cases for $n = 0$ and $n = 1$, which stop further recursive calls.
2. For values greater than 1, the function returns the sum of the two preceding Fibonacci numbers, and the result is displayed for a given position.

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

observations: 1.The `read_and_process_file()` function reads numbers from a file, converts each line into an integer, and calculates their average, while checking for empty files or invalid data.

2.The function uses structured exception handling to manage file-not-found errors, value conversion errors, and unexpected exceptions, ensuring the program terminates safely with clear messages.

```

Lab5.task5 > @ Task10.py > read_and_process_file
1 #Generate a well commented code with proper error handling and clear explanations for each exception.
2 def read_and_process_file(filename):
3     """
4     This function reads numbers from a file and calculates their average.
5     Each number should be on a separate line.
6     """
7     try:
8         # Attempt to open and read the file
9         file = open(filename, "r")
10        lines = file.readlines()
11        file.close()
12        numbers = []
13        # Convert each line into an integer
14        for line in lines:
15            number = int(line.strip()) # May raise ValueError
16            numbers.append(number)
17        # Check if file is empty
18        if len(numbers) == 0:
19            raise ValueError("File is empty. No data to process.")
20        # Calculate average
21        average = sum(numbers) / len(numbers)
22        print("Numbers in file:", numbers)
23        print("Average:", average)
24    except FileNotFoundError:
25        # Happens if the file does not exist
26        print("Error: File not found. Please check the file name.")
27    except ValueError as ve:
28        # Happens if conversion to int fails (if file is empty)
29        print("Error:", ve)
30    except Exception as e:
31        # Catches any unexpected error
32        print("Unexpected error occurred:", e)
33    finally:
34        # Always runs whether error occurs or not
35        print("File processing completed.")
36
37 # Example usage
38 filename = "data.txt"
39 read_and_process_file(filename)
40

```

```

PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task10.py
Error: File not found. Please check the file name.
File processing completed.
PS D:\courses\AIAC>

```